An Automated BB84 Protocol with Single Heralded Photons

Joseph Quinn, Dr. Markus Allgaier, The University of North Dakota

UNIVERSITY OF NORTH DAKOTA
COLLEGE OF ARTS & SCIENCES

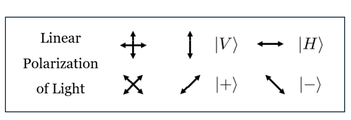
Undergraduate Showcase

Introduction

- The BB84 Protocol is a quantum key distribution protocol used for encryption. It requires single photon sources to properly function.
- The goal of this experiment is to automate a BB84 protocol with single heralded photons.

The BB84 Protocol with Single Heralded Photons

The BB84 Protocol utilizes four linear polarization states of photons in two bases, A rectilinear basis, $|H\rangle + |V\rangle$, and a diagonal basis, $|+\rangle + |-\rangle$.



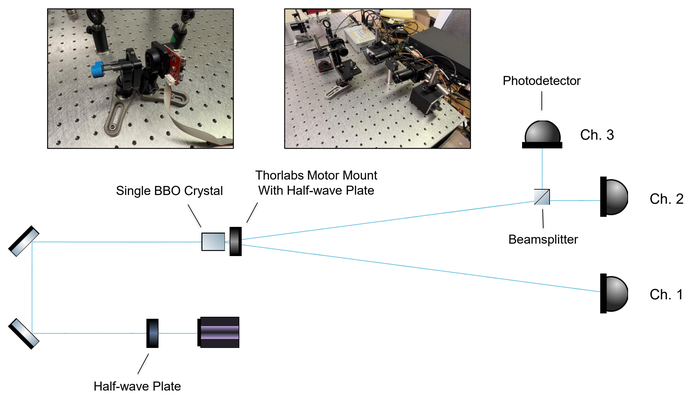
Alice and Bob communicate between each other on a quantum and classical channel. Alice sends photons to Bob on the classical channel, and Bob chooses one of the two bases to measure in.



For both bases, one of the photons represents a 0 bit and the other represents a 1 bit. Many measurements are collected, and then Alice and Bob communicate with each other over the classical channel to determine which of their bits match. The matching bits become a sifted key, which is used for further communication. An eavesdropper  can be detected if while using the sifted key it is observed that some of the bits do not match.

For the BB84 Protocol to work, single photon sources are needed. Single photon sources are hard to make, so, instead, the single photon is “heralded” via another single photon in a photon pair, commonly through Spontaneous Parametric Down Conversion (SPDC) or Spontaneous Four Wave Mixing (SFWM). I use SPDC for my experiment.

Experimental Setup



Automation



Time Tagger 20, Swabian Instruments



ELL16K Motor Mount, ThorLabs

```python
start_time = time.time()
elapsed_time = 0
i = 0


while i < number_of_bits:

    counts = [0., 0., 0., 0.]

    controller = elliptec.Controller('COM4')
    ro = elliptec.Rotator(controller)
    random_number = random.randint(0, 100)
    rand_mod_4 = random_number % 4
    pol_angle = rand_mod_4*45
    print(pol_angle)
    ro.set_angle(pol_angle)
    controller.close_connection()


    elapsed_time = time.time() - start_time


    while counts[2] < 2.0 and counts[3] < 2.0:
        print(f"Counts: {counts}")
        rate.startFor(measurementDuration, clear=True)
        rate.waitUntilFinished()
        counts += (rate.getData())/(1e3)
        print(f"Counts: {counts}")
        rate.stop()


        if counts[2] >= 2.0 and counts[3] >= 2.0:
            break
        elif counts[2] == 1.0:
            bits = np.append(bits, 0)
            polarizations = np.append(polarizations, pol_angle)
            print(f"Bits: {bits}")
            i += 1
            break
        elif counts[3] == 1.0:
            bits = np.append(bits, 1)
            polarizations = np.append(polarizations, pol_angle)
            print(f"Bits: {bits}")
            i += 1
            break
```

Main portion of the code for automation

```
Bits: [1. 0. 1. 0. 0. 0. 0. 0. 1. 1. 0. 1. 0. 0. 0. 1. 0. 0. 0. 1. 0. 1. 0.
 1.
 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 0. 1. 1. 1. 1. 1. 1. 0. 1. 0. 0. 0. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 0. 1. 1. 0. 1. 1. 0. 1. 1. 1. 0. 1. 1. 1. 0.
 1. 0. 1. 1. 1. 1. 1. 0. 0. 1. 1. 0. 1. 1. 1. 1. 1. 0. 1. 0. 0. 0. 1. 1. 1. 1. 0.
 0. 1. 1. 1. 0. 0. 1. 1. 1. 1. 0. 0. 1. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0.
 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 0. 0.
 1. 1. 0. 0. 0. 1. 0. 1. 1. 1. 0. 1. 1. 0. 1. 0. 1. 1. 1. 0. 0. 0. 0. 1. 1. 1.
 0. 1. 0. 1. 1. 1. 0. 1. 1. 1. 1. 1. 0. 0. 1. 1. 0. 1. 1. 1. 0. 0. 0. 1. 0. 0.
 0. 0. 0. 0. 0. 1. 1. 0. 0. 1. 1. 0. 1. 0. 1. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0.
 0. 1. 1. 1. 0. 1. 0. 1. 1. 1. 0. 1. 0. 0. 0. 1. 1. 0. 0. 0. 1. 0. 0. 1.
 1. 0. 1. 1. 0. 1. 1. 1. 0. 1. 0. 1. 1. 1. 1. 1. 0. 1.]

~~~ There were 256 bits created in 208 seconds. ~~~
```

The output

Experimental Overview

In this experiment, the motor mount half-wave plate can be treated as Alice, and the beam splitter can be treated as Bob. For the protocol, the half-wave plate encodes photons into four different states in two bases, but the beamsplitter only gives two outcomes, sending a photon to `channel 2` for a horizontal polarization, and a photon to `channel 2` for a vertical polarization. The former is represented by a 0 bit, and the latter a 1 bit. The single photon that goes to `channel 1` is the “herald” photon of the photon that goes through the beamsplitter.

To automate the experiment, I connect to the Time Tagger 20 and the ThorLabs motor mount through their python libraries. For a given number of bits, the motor mount moves to a random polarization, and the Time Tagger measures the coincidences (photon pairs) between channels 1 and 2 (0 bit), and channels 1 and 3 (1 bit). The bits are then appended to a `bits` array along with the polarization that was used by the half-wave plate for each. Since the beam splitter (Bob) is measuring only in the rectilinear basis, all photons sent by the diagonal basis are discarded, forming a sifted key.

Conclusions

I was successful in automating the BB84 protocol with single heralded photons using the Time Tagger 20, a ThorLabs motor mount, and other lab equipment.

Acknowledgements

A special acknowledgement to Dr. Markus Allgaier for helping me with my experimental setup and code.

References

- SujayKumar et al. (2025). Comprehensive Analysis of BB84, *A Quantum Key Distribution Protocol. Vellore Institute of Technology.*